Vulnerability Report: Insecure Answer Validation in Rosetta Stone Platform

1. Introduction

In this report, we describe a security vulnerability observed in the Rosetta Stone language learning platform used by université cadi ayyad. By circumventing the intended learning process, certain users can bypass prior answers and display the one that they might have provided using answer validation principles, though it is always determined by manipulations in client-side settings. Such attacks occur because the server-side only validates submitted answers lightly while ensuring front-end logic is largely exercised for correctness checks.

2. Vulnerability Details

- **Platform:** Rosetta Stone Language Learning Platform.
- **Vulnerability Type:** Insecure Answer Validation / Client-Side Validation Bypass
- **Location:** Answer submission process via GraphQL API.
- **Affected Parameters :**
    - *score* (Float): Represents the correctness score of the answer. Client-side typically sets 1.0 for correct, less than 1.0 for incorrect/partial.
    - *skip* (Boolean): Indicates if the question was skipped. Client-side sets true for skipped, false for answered.
- **Description:** Description: Rosetta Stone is a React application that depends almost exclusively on JavaScript front-end code to check whether the user's answer is correct. When an answer is sent, a GraphQL API request is made to the backend. This request carries with it score and skipped parameters determined by the front-end logic.

  Crucially, the backend appears to lack sufficient server-side validation of these parameters. An attacker, A burp suite can intercept the GraphQL request and change score and skipped values to force the backend into marking any answer as correct, no matter what the actual user input or front-end's first judgment was.

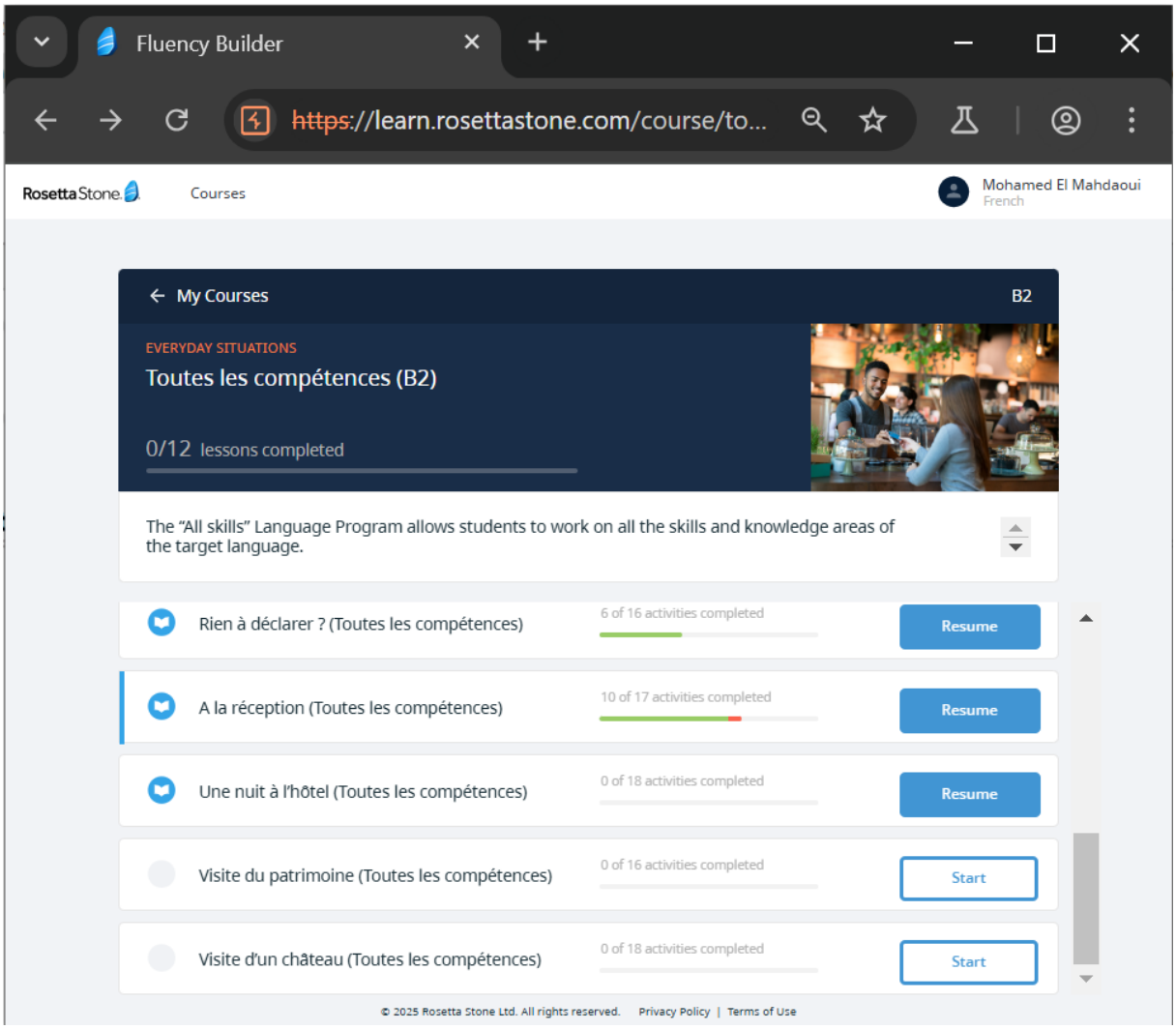
  Even though the front sees that the answer is marked as wrong according to its local logic, on refreshing the page, or up to a backend resync, the platform would show the manipulated data and mark the question correct, hence updating progress metrics accordingly.

3. Proof of Concept

1. **Prerequisites:**
    - Access to the Rosetta Stone platform through [University Name]'s account.
    - Burp Suite or a similar web interception proxy tool installed and configured.
    - Basic understanding of using Burp Suite to intercept and modify HTTP requests.
2. **Steps:**

a. Log in to your Rosetta Stone account and navigate to a lesson with interactive exercises.

b. Start an exercise and reach a question requiring an answer.

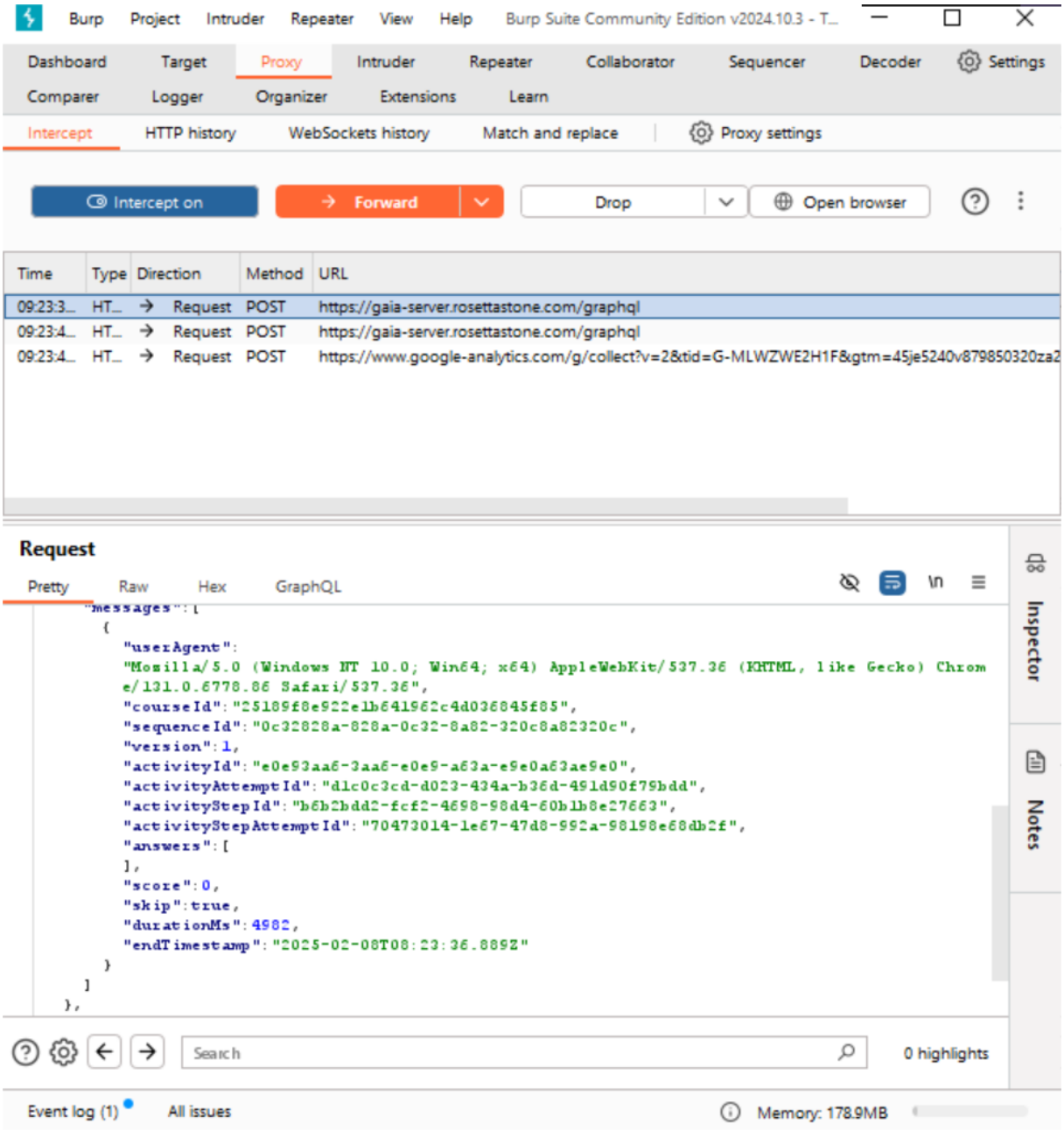


c. **Enable Burp Suite interception.**

d. Answer the question incorrectly (or even leave it blank).

e. Submit the answer within Rosetta Stone.

f. **In Burp Suite, intercept the POST request to the GraphQL endpoint.** This will likely be to a URL containing /graphql or similar. Look for a request that appears to be related to answer submission (inspect the request body).

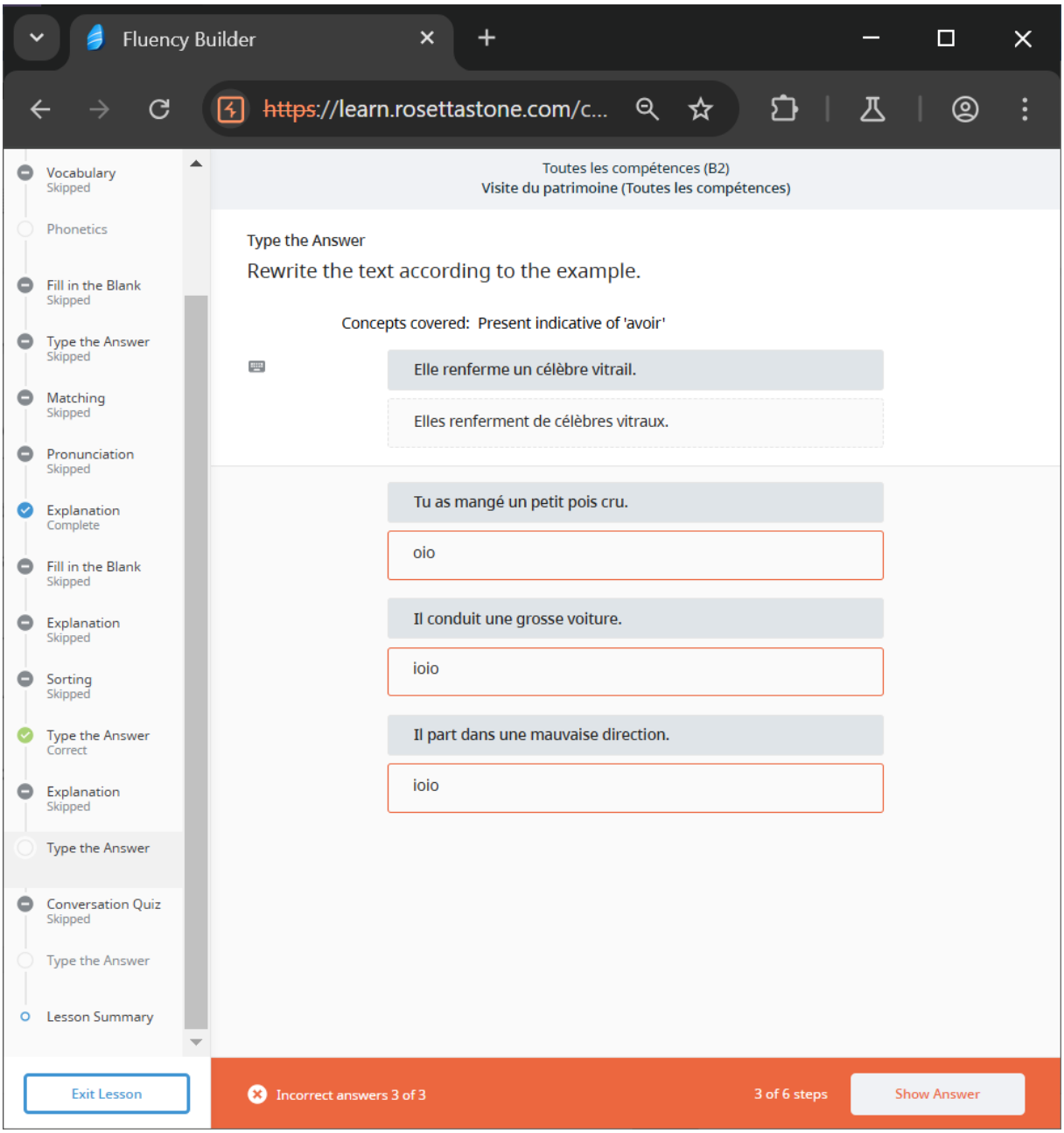


g. **Examine the request body.** You should find the score and skip parameters within the GraphQL query or mutation.

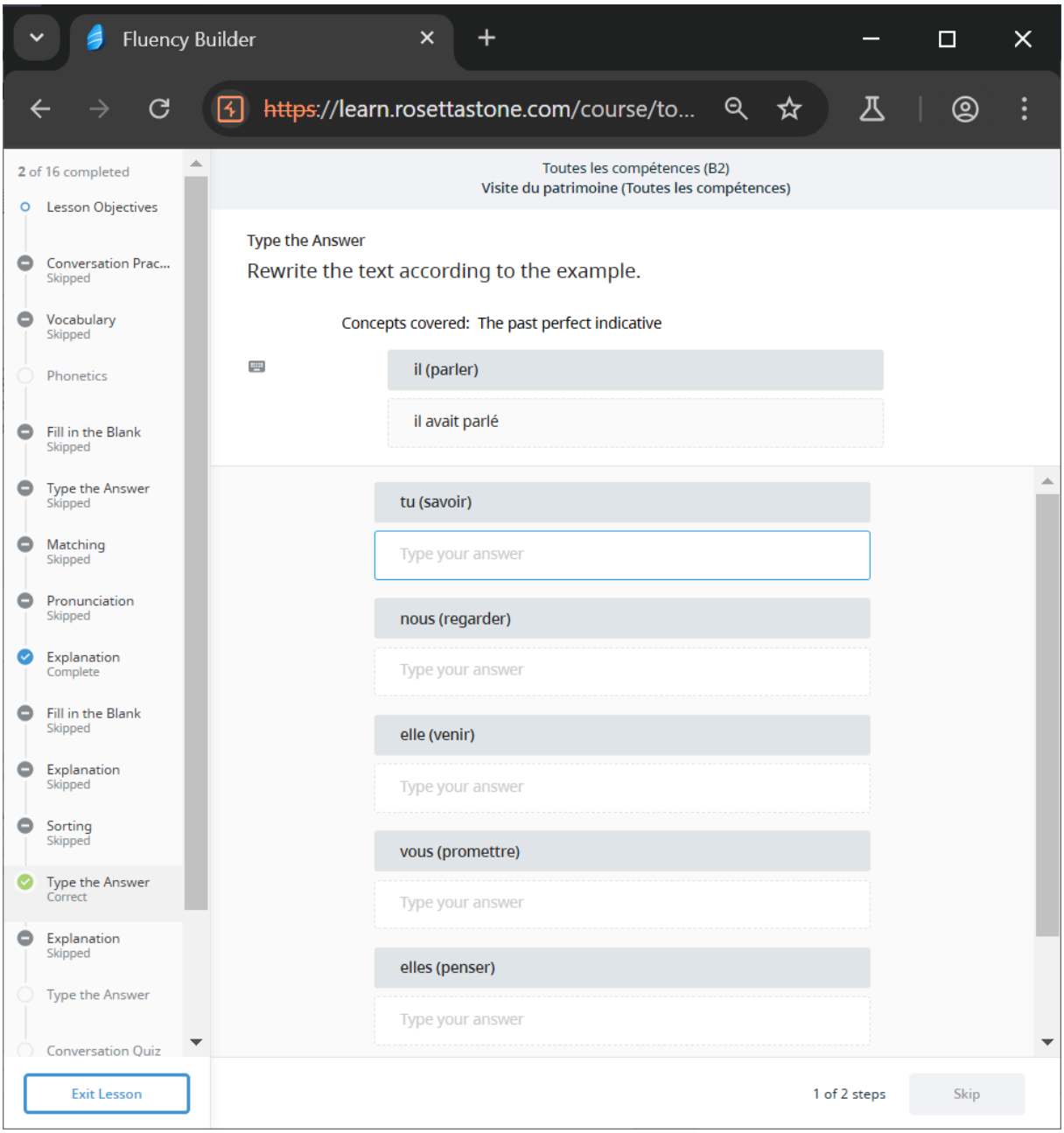
h. **Modify the parameters.** Change the following: * Set score to 1.0 (or a value indicating full correctness, if you see different values in other requests). * Set skipped to false.

i. **Forward the modified request** in Burp Suite to the server.

j. **Observe the Rosetta Stone interface.** Initially, the front-end may still show the answer as incorrect or skipped.



k. **Refresh the page** within Rosetta Stone, or navigate away and back to the lesson.



l. **Verify the result.** The question should now be marked as correct, and your progress should be updated accordingly.

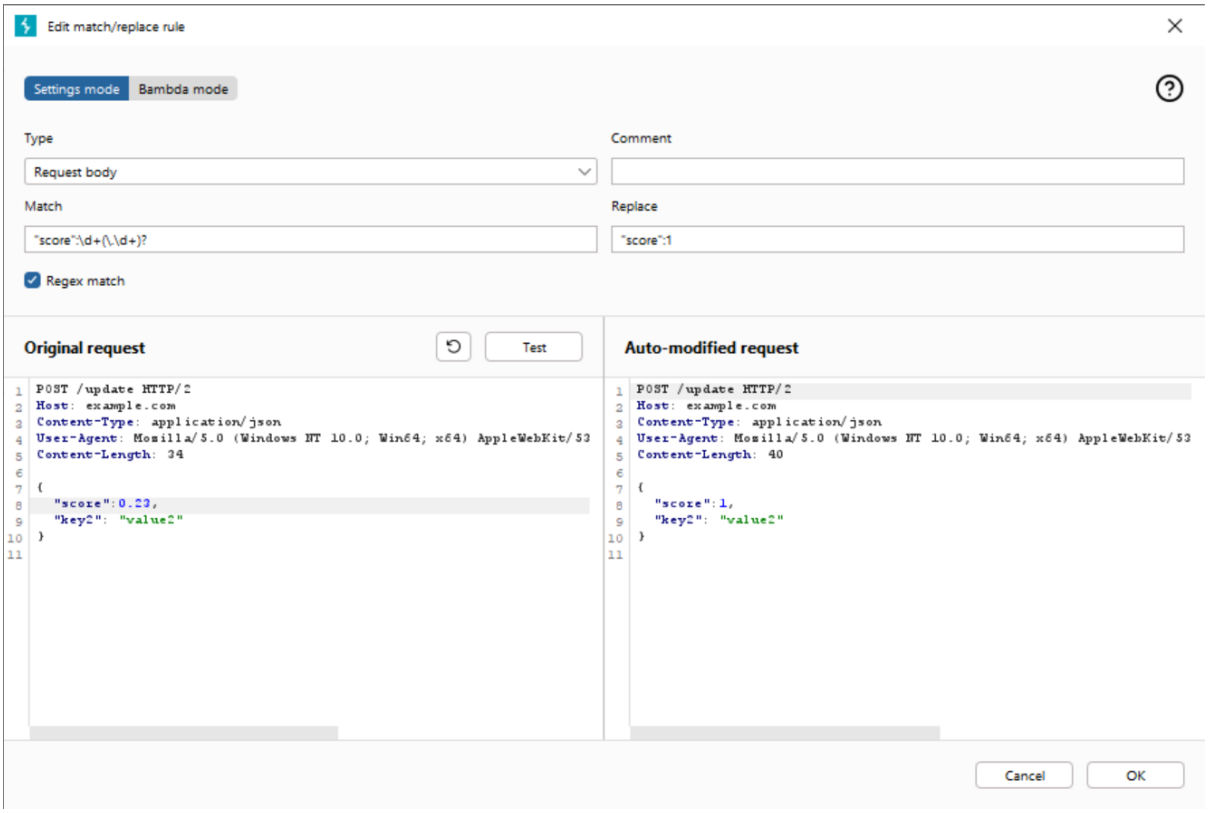
Automation (Using Burp Suite Match and Replace)

This vulnerability can be easily automated using tools like Burp Suite's "Match and Replace" functionality. This approach allows for real-time modification of requests without requiring custom scripting. Here's how to automate the exploitation using Burp Suite Match and Replace rules:

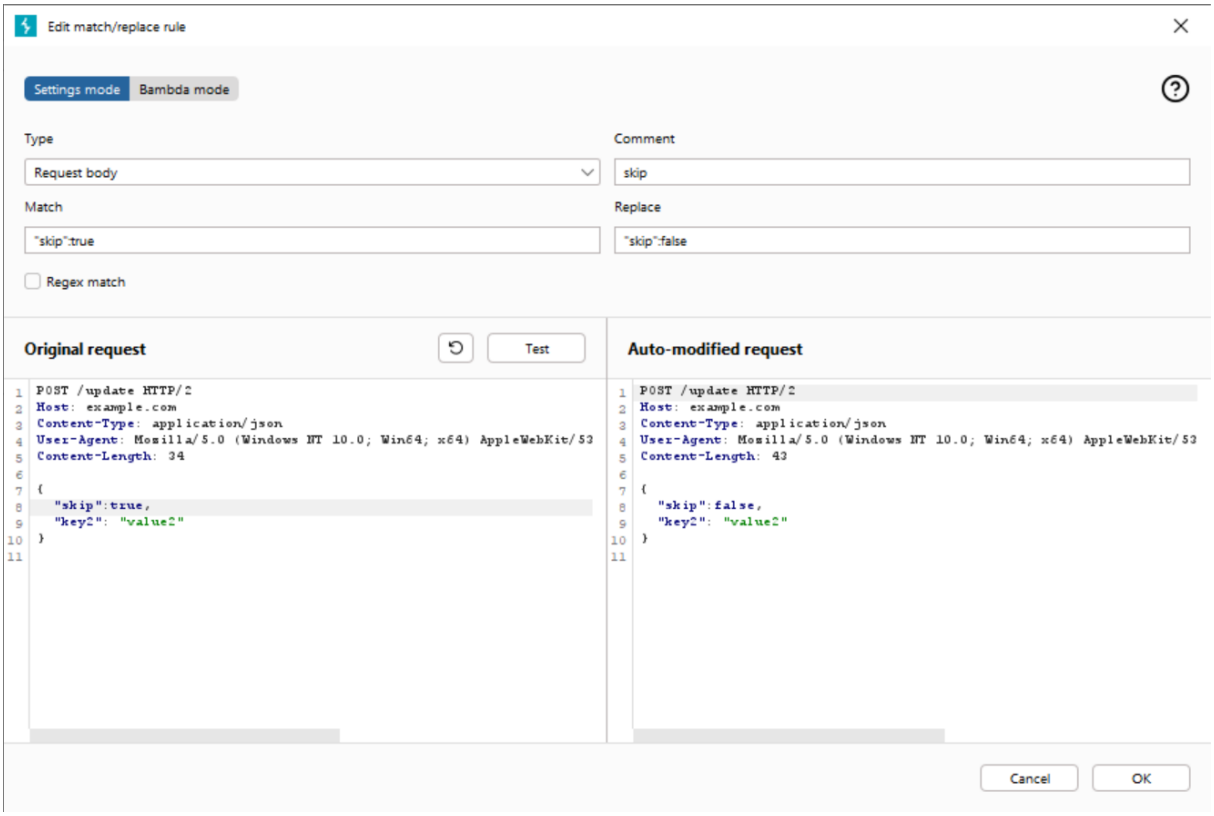
1. **Access Burp Suite's Match and Replace settings:** Navigate to "Proxy" -> "Options" -> "Match and Replace" tab.
2. **Create Match and Replace Rules:** You can create separate rules for each parameter you want to modify (score, skipped, and potentially correct if that's also relevant based on your

observations - though score and skipped are likely sufficient). Use the "Add" button under "Request body".

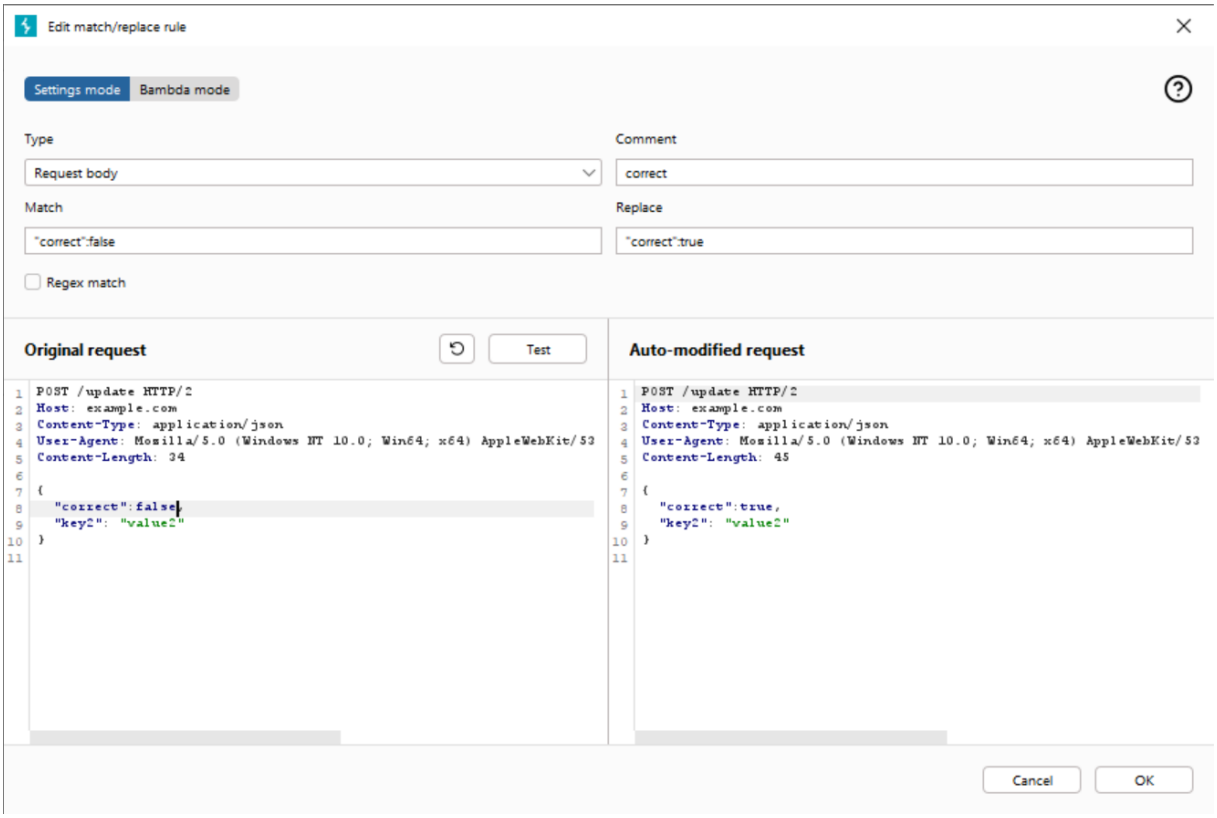
3. **Configure Rules for score:**
    - **Type:** "Request body"
    - **Match:** Use a regular expression to dynamically capture the existing score value. For example, if the score parameter is consistently formatted like "score":0.23, you could use the regex: "score":\d*\.?\d* (This regex matches "score": followed by digits, optionally a decimal point, and more digits).
    - **Replace:** "score":1 (or "score":1.0 depending on the expected format)
    - **Comment:** "Force Correct Score"
    - **Enable Regex match:** Check this box.



4. **Configure Rules for skipped:**
    - **Type:** "Request body"
    - **Match:** "skipped":true (or "skipped":false if you want to reverse the logic and always mark as not skipped, regardless of the original value)
    - **Replace:** "skipped":false
    - **Comment:** "Force Not Skipped"



5. **Configure Rules for correct :**
    - **Type:** "Request body"
    - **Match:** "correct":false (or "correct":true to reverse logic)
    - **Replace:** "correct":true
    - **Comment:** "Force Correct Answer"



6. **Enable the Rules:** Ensure all created Match and Replace rules are enabled (checkbox checked).

Thus, by having these rules active within Burp Suite, any answer submission request that you will forward through Burp will cause the parameters specified to be modified in real-time to reflect that the answer is indeed correct in the backend. This allows a much faster and convenient means of working through Rosetta Stone exercises without actually answering the questions correctly.

5. Potential Impact

- **Circumvention of Learning:** Students can easily bypass the intended learning process and complete courses without genuinely engaging with the material. This undermines the effectiveness of Rosetta Stone as a learning tool.
- **Inaccurate Progress Tracking:** The platform's progress tracking and reporting become unreliable, as users can artificially inflate their scores and completion rates.
- **University Resource Misallocation:** If widespread, this vulnerability could lead to inaccurate data regarding the platform's usage and effectiveness, potentially misguiding university decisions about resource allocation and platform selection.
- **Ethical Concerns:** While primarily a technical vulnerability, it raises ethical concerns about academic integrity if students exploit this to falsely represent their language proficiency.

7. Responsible Disclosure

This vulnerability was discovered during academic exploration and out of curiosity regarding the platform's security. There was no exploitation for personal gain, and never has this information been relayed to the outside world or shared with any fellow classmates.

N.B.: To enhance the clarity, language level, and formality of this report, AI language models were utilized for text refinement and editing only. The technical findings, analysis, and report structure are entirely the work me.